

TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation

Team : Fundamental
Presenter: Kim Min Jung

1. Introduction

Limitations of Existing Vision-Language-Action (VLA) Models and Research Questions

Emergence of VLA Models

- Vision-Language-Action (VLA) models learn end-to-end mappings from **images (vision) + text instructions** to **robot actions**.
- Models such as **RT-2** and **OpenVLA** have demonstrated strong performance in multitasking and generalization.

Limitations of Existing VLA Models

1. Inference Speed Issues

- They rely on **large vision-language models ($\geq 7B$ parameters)**.
- Actions are generated in an **autoregressive next-token manner**, producing tokens sequentially for each degree of freedom → **high latency** during inference.

2. Training Cost Issues

- Models like OpenVLA require large-scale robot datasets (e.g., **OpenX: 970k samples**) for pretraining.
- This leads to **significant computational cost and resource consumption**, making deployment in real robots or real-world services difficult.

How can we design a VLA model that retains the performance and generalization benefits of existing VLAs, while achieving faster inference and improved data efficiency?

1. Introduction

TinyVLA: A Fast and Data-Efficient VLA Model

One-line takeaway

= lightweight VLM backbone + diffusion policy head → fast inference & strong generalization without massive robot pretraining

Core Ideas (3 Pillars)

1. Lightweight VLM Backbone (<1B params)

Compact but strong multimodal backbone for efficient inference

2. Diffusion-Based Policy Head (continuous actions)

Replaces autoregressive action-token generation

Directly predicts continuous robot actions (e.g., 6D pose)

3. Generalization without Large-Scale Robot Pretraining

No reliance on massive robot datasets (e.g., OpenX)

Leverages world knowledge from vision+language pretraining

Main Contributions (Short)

1. **New VLA architecture:** lightweight VLM + diffusion head
→ **fast + strong + robust** simultaneously
2. **Extensive evaluation:**
simulation + real-world, single-arm + bimanual
→ demonstrates broad applicability
3. **Evidence of data efficiency**
→ strong VLA models can be trained
without massive robot datasets

1. Introduction

TinyVLA-H vs. OpenVLA

Multitasking Performance

- Across both simulation and real-world Franka robot settings, **TinyVLA significantly outperforms the baseline (OpenVLA)** in multitask performance.

Real-World Single-Arm Franka Experiments

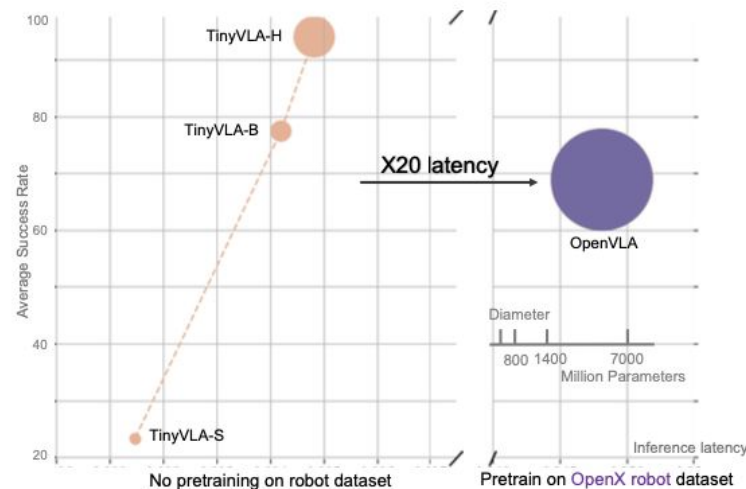
- TinyVLA-H achieves a +25.7 percentage point higher success rate** compared to OpenVLA.
- Uses **5.5× fewer parameters** than OpenVLA.

Bimanual Robot Experiments

- OpenVLA** shows degraded performance on bimanual tasks because **OpenX contains only single-arm demonstrations**. **TinyVLA-H** maintains strong performance and **substantially outperforms OpenVLA** even under this condition.

Inference Speed

- Inference Latency vs. Average Success Rate* On the same A6000 GPU, **TinyVLA-H achieves approximately 20× lower inference latency** than OpenVLA **while achieving higher average success rates**.
- Bubble size represents model parameter count → visualizing the fact that TinyVLA-H is **smaller, faster, and performs better**.



2. Related Work

OpenVLA: An Open-Source Vision-Language-Action Model

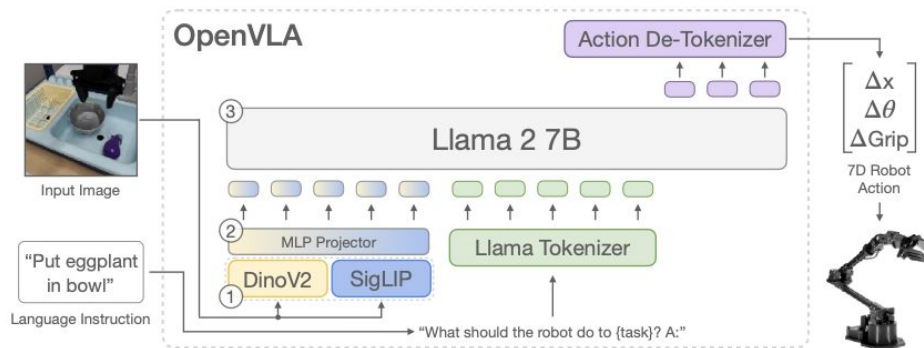


Figure 2: **OpenVLA model architecture.** Given an image observation and a language instruction, the model predicts 7-dimensional robot control actions. The architecture consists of three key components: (1) a **vision encoder** that concatenates Dino V2 [25] and SigLIP [79] features, (2) a **projector** that maps visual features to the language embedding space, and (3) the **LLM backbone**, a Llama 2 7B-parameter large language model [10].

1. **Input: Vision–Language Fusion SigLIP** (semantic features) and **DINOv2** (spatial features) visual encoders process the image.
 - The extracted visual features are passed to **Llama 2 (7B)** through an **MLP projector**.
2. **Action Discretization** : Continuous robot motions (position, rotation, etc.) are discretized into **256 predefined bins**.
3. **Integer Encoding** : Each bin is mapped to a **discrete integer value from 0 to 255**, representing robot actions.
4. **Token Substitution** : **256 low-frequency tokens** in the Llama 2 vocabulary are replaced with these discrete integers, forming **action tokens**.
5. **Output: Next Action Prediction**
 - Given the visual and language context, **Llama 2 predicts the most probable next action token**.
 - The robot executes the motion corresponding to the predicted token.

2. Related Work

LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Problems with Conventional Fine-tuning

- Updates the entire pretrained weight matrix W_0
- Parameter count, memory usage, and training cost all increase
- Infeasible for large-scale models

Core Idea of LoRA

“Don’t train the entire weight matrix—train only the change.”

When large models learn a new task, the amount of information that actually needs to change is very small. Instead of updating all parameters, it is sufficient to learn a **low-rank update** that captures this change.

$$W = W_0 + \Delta W$$

$$\Delta W = BA$$

W_0 : pretrained weight (**freeze**)

$$B \in \mathbb{R}^{d \times r}$$

$$A \in \mathbb{R}^{r \times k}$$

$r \ll d, k$ (low-rank)

Matrix Decomposition

Frozen Path (blue): The original weight matrix W_0 is kept fixed and not updated during training.

Trainable Path (orange): The weight update ΔW is decomposed into two small matrices B and A and trained, with rank $r \ll d$

Forward Pass: The outputs from the two paths are summed. (h)

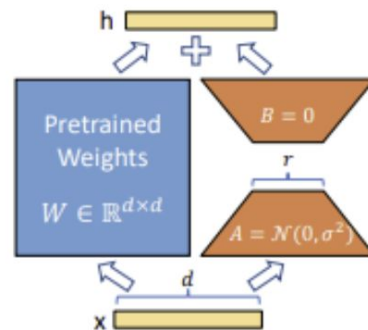
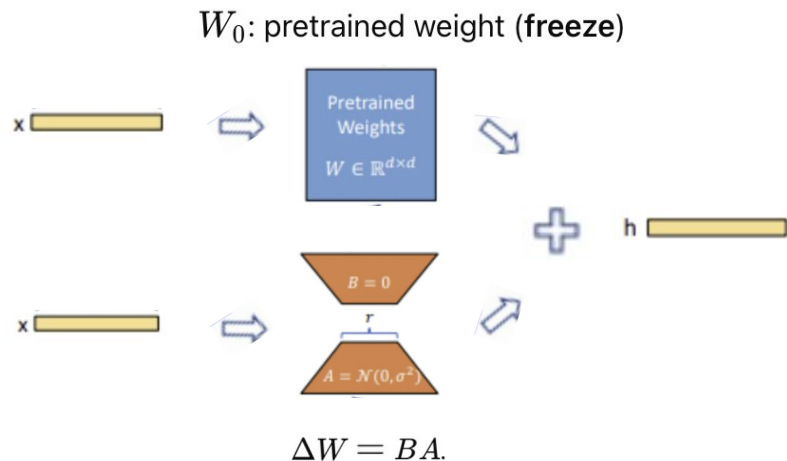


Figure 1: Our reparametrization. We only train A and B .

2. Related Work

LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS



Forward Structure

$$W = W_0 + \Delta W = W_0 + BA.$$

$$h = W_0 x + \Delta W x$$

$$= \underbrace{W_0 x}_{\text{Pretrained Model Output}} + \underbrace{BAx}_{\text{Task-specific Adjustment (Term)}}$$

Pretrained Model Output

Task-specific Adjustment (Term)

Where Do We Apply LoRA?

Core weight matrices in **Self-Attention**:

- W_q (Query) **Query**: Determines *where to attend*
- W_v (Value) **Value**: Determines *what information to retrieve*

→ Directly controls the attention mechanism, achieving maximum efficiency

Key Characteristics Summary

$B=0$ initialization → the model starts fully pretrained

Very small ranks (e.g., $r=1-4$) are sufficient

During inference, W_0+BA can be **merged**

→ no increase in inference latency

2. Related Work

Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

1. Iterative Refinement Process (The Process)

- $\mathbf{O}$ (Observation): The current state perceived by the robot through its camera.
- $\mathbf{a}$ (Action): The action, which is initially in a random noisy state.
- $\varepsilon_{\theta}(\mathbf{O}, \mathbf{A})$
Given the current state (**observation**) and the noisy action (**action**), the network identifies the noise component mixed into the action and computes **the direction in which the action should be corrected to become the optimal behavior**.
- $\mathbf{K}$
Rather than producing the correct answer in a single step, the action is gradually refined by repeatedly applying small corrections in the computed direction for **$\mathbf{K}$ iterations**.

2. Gradient Field

- Blue arrows:** The gradient indicating how the action should be adjusted at each step.
- Black curve:** The target action trajectory representing the correct behavior.

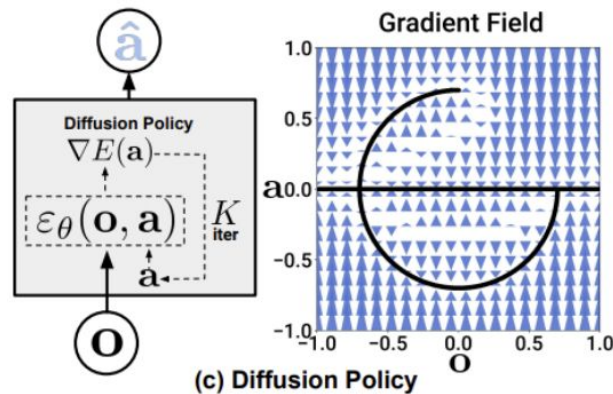


Figure 1. Policy Representations.

2. Related Work

Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

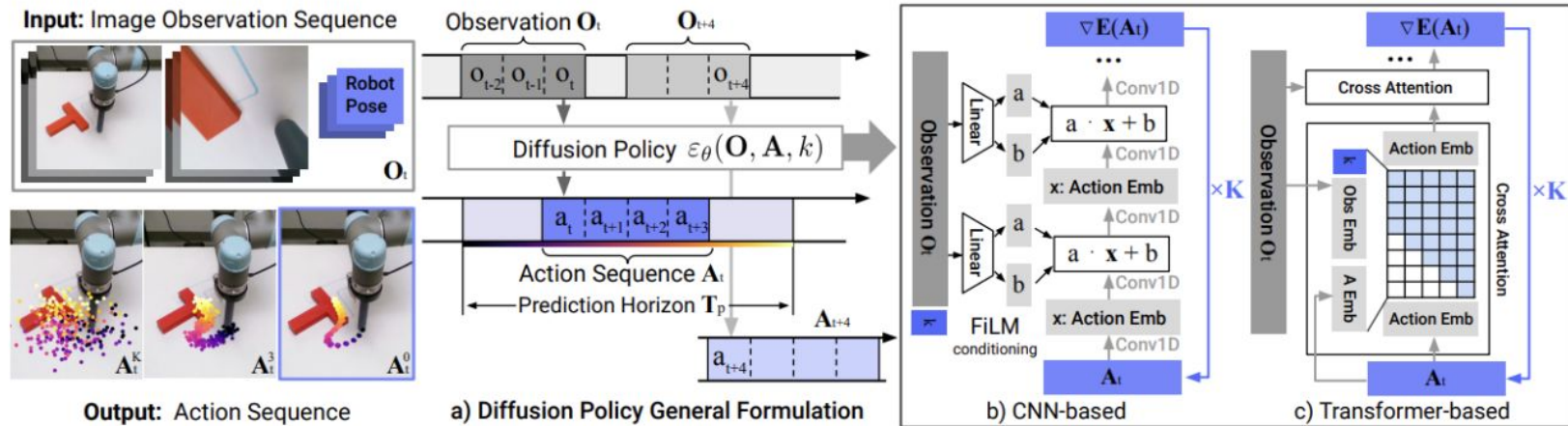


Figure 2. Diffusion Policy Overview a) General formulation. At time step t , the policy takes the latest T_o steps of observation data O_t as input and outputs T_a steps of actions A_t . b) In the CNN-based Diffusion Policy, FiLM (Feature-wise Linear Modulation) Perez et al. (2018) conditioning of the observation feature O_t is applied to every convolution layer, channel-wise. Starting from A_t^K drawn from Gaussian noise, the output of noise-prediction network ε_θ is subtracted, repeating K times to get A_t^0 , the denoised action sequence. c) In the Transformer-based Vaswani et al. (2017) Diffusion Policy, the embedding of observation O_t is passed into a multi-head cross-attention layer of each transformer decoder block. Each action embedding is constrained to only attend to itself and previous action embeddings (causal attention) using the attention mask illustrated.

3. Method

TinyVLA consists of three core designs:

Pretrained VLM as Backbone

- Uses a pretrained multimodal model (VLM) to initialize the policy network
- Leverages strong priors for scene understanding and generalization, even without robot-specific data

Parameter-Efficient Fine-tuning (LoRA)

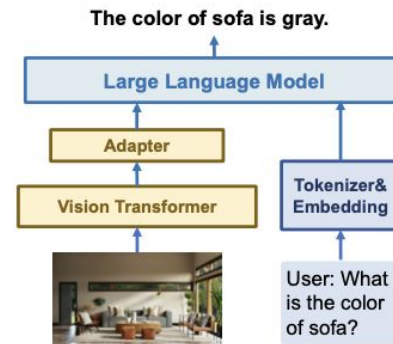
- Freezes the pretrained backbone during robot data training
- Trains only a small subset of parameters via LoRA → Trainable parameters $\approx 5\%$

Policy Decoder for Actions

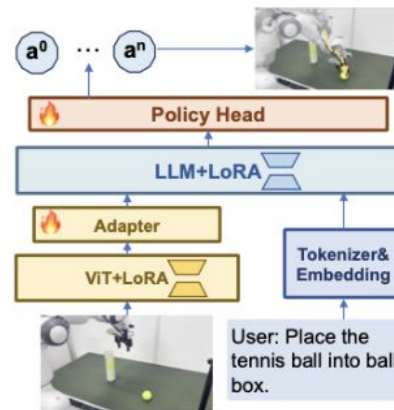
- VLM outputs are connected to the policy decoder through a **linear projection**
- Final output: **continuous actions** that the robot can execute

Why a Lightweight VLM?

- Conventional VLA/VLMs typically rely on large multimodal models (3B+ parameters) → slow inference and high computational cost
- **TinyVLA** instead uses a **lightweight VLM** in the range of **70M–1.4B parameters**, trained and deployed directly



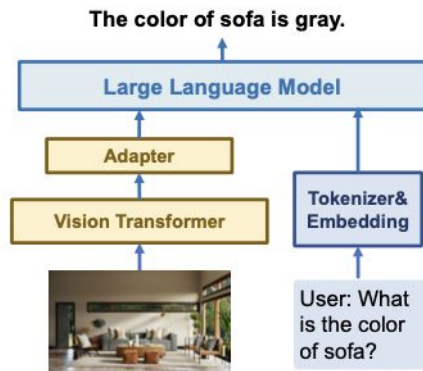
VLM Pretraining



Policy Finetuning

3. Method

A. Building TinyVLA with Efficient Vision-Language Models



VLM Pretraining

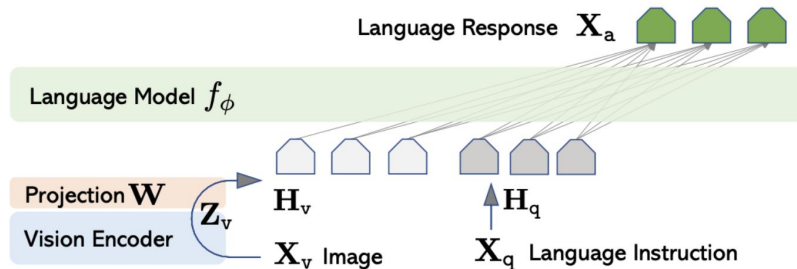


Figure 1: LLaVA network architecture.

How the VLM Is Built

Language Backbone : Based on **Pythia** → Lightweight alternative to GPT/LLaMA-scale models

Training Pipeline : Follows **LLaVA** → Uses image-text (vision-language) datasets for multimodal alignment

LLaVA: Model Architecture

Language model: Vicuna, fine-tuned from LLaMA

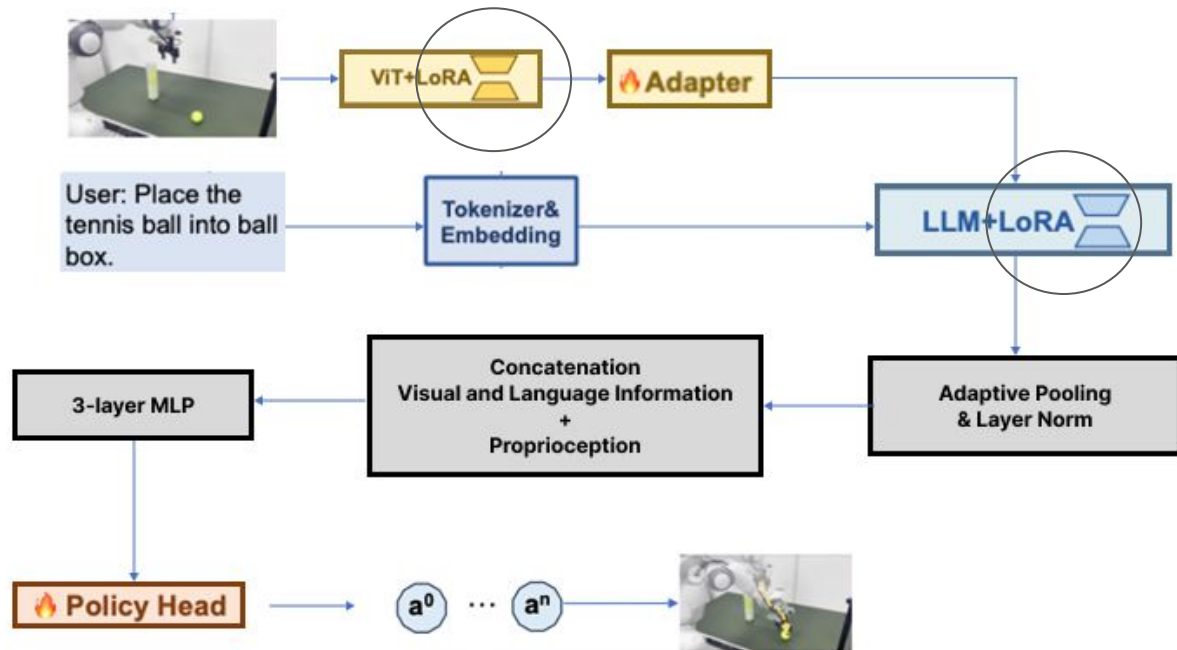
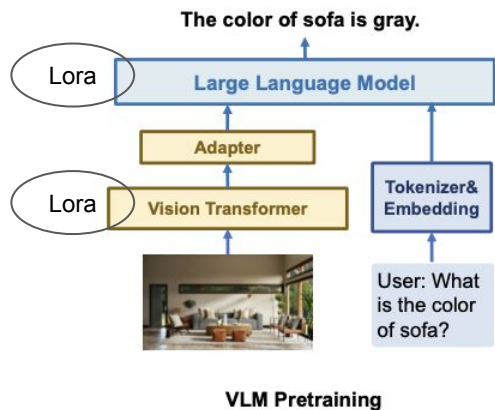
Vision encoder: CLIP visual encoder (ViT-L/14)

Projection WWW: A simple linear layer that maps image features into the word embedding space

3. Method

B. Robot Data Finetuning for Manipulation

Frozen weights and low-rank adaptation.



TinyVLA predicts 7-dimensional actions, including position (x, y, z), rotation (roll, pitch, yaw) and (**gripper_width**).

4. Experiments

A. Experimental Setup

1. Model Variants

Experiments are conducted with three TinyVLA variants based on model size:

TinyVLA-S (Small), **TinyVLA-B (Base)**, and **TinyVLA-H (Huge)**.

2. Simulation Benchmark

- **Environment:** MetaWorld benchmark (50 tasks with four difficulty levels).
- **Setting:** Multi-task learning using **50 demonstration trajectories**.

3. Real Robot Evaluation

- **Hardware:**
 - *Single-arm:* Franka Panda robot (2 cameras)
 - *Bimanual:* Two UR5 arms (3 cameras)
- **Tasks:** Ranging from simple stacking to complex manipulation such as **FlipMug** and bimanual coordination tasks.
- **Data:** 100 teleoperated demonstrations per task; predicts **7-DoF actions** (xyz position + 3-axis rotation + gripper).

4. Baselines

- **Models:** Diffusion Policy, Multimodal Diffusion, OpenVLA.
- **Fair comparison:**
 - OpenVLA adapted to **multi-view** inputs
 - Diffusion Policy enhanced with **language conditioning (FiLM)**
→ All baselines are upgraded to match the same experimental conditions.

4. Experiments

B. Experimental Results on Multi-Task Learning

Simulation experimental results

Model \ Tasks	Metaworld (50 tasks)				Avg.
	Easy (28)	Medium (11)	Hard (6)	Very Hard (5)	
Diffusion Policy [3]	23.1	10.7	1.9	6.1	10.5
TinyVLA-H	77.6	21.5	11.4	15.8	31.6

TABLE I: Comparing TinyVLA with Diffusion Policy in **simulation**. We report the average success rate on multiple tasks, We use TinyVLA-H as our method. **All methods are trained in a multi-task setting.**

Real-world experimental results

Model \ Tasks	Pre-trained Trajectory	Total Params	Trainable Params	RealWorld(5 tasks)					Avg.
				PlaceTennis	FlipMug	StackCubes	CloseDrawer	OpenBox	
Diffusion Policy [3]	N/A	111M	111M	16.7±0.6	30±0.2	3.3±0.1	73.3±0.1	53.3±0.1	35.3
Multimodal Diffusion [38]	N/A	230M	230M	23.3±0.3	13.3±1.3	6.7±0.3	36.7±0.3	10.0±0	18.0
OpenVLA [10]	970K	7.2B	195M	83.3±1.1	51.7±3.1	40.0±0.1	85.0±1	81.7±0.6	68.3
TinyVLA-S	N/A	422M	101M	8.3±0.1	6.7±0.1	6.7±0.1	60.0±0.2	35.0±0.3	23.3
TinyVLA-B	N/A	740M	138M	76.7±0.6	76.7±0.1	71.7±0.1	81.7±0.1	80.0±0.2	77.4
TinyVLA-H	N/A	1.3B	143M	90.0±0.2	98.3±0.1	98.3±0.1	96.7±0.3	86.7±0.1	94.0

TABLE II: Quantitative results in real-world experiments.

We report the average success rate across multiple tasks and the count of trainable parameters for all models.

4. Experiments

C. Generalization to Unseen Instructions

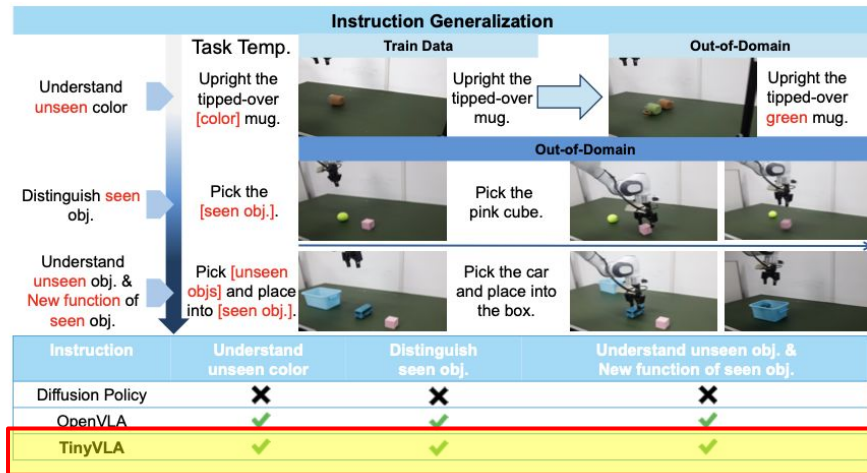


Fig. 4: **Instruction Generalization.** We conducted three different types of instruction generalization experiments with progressively increasing difficulty.

D. More Real-World Experiments: Bimanual Robot

TABLE III: **Quantitative results for bimanual UR5 real robot experiments.** We report the average success rate over 10 trials. All models are trained in multi-task settings.

Model \ Tasks	Trainable Params	RealWorld(3 tasks)		
		PlaceBread	StackCubes	PlaceTennisBag
DP [3]	111M	40.3±1.7	31.3±1.3	43±2.3
OpenVLA [10]	195M	0±0	0±0	0±0
TinyVLA-H	143M	76.7±2.3	36.7±2.3	30±1

4. Experiments

E. Experiments on Generalization

[Generalization to new views]

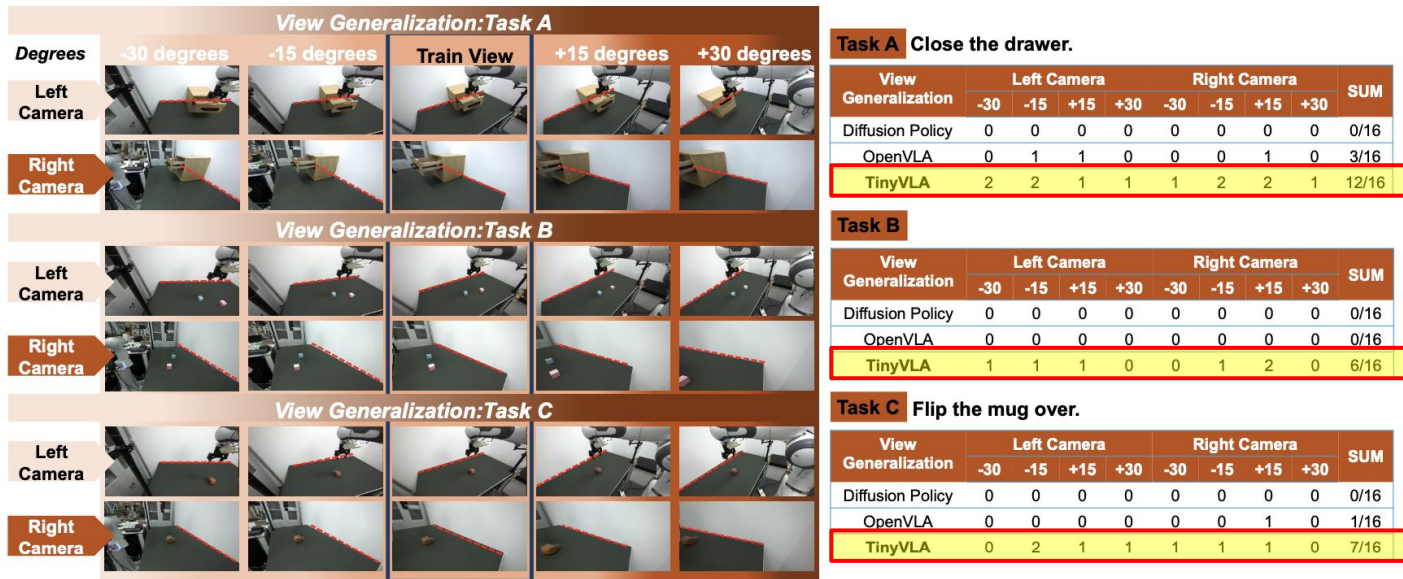


Fig. 5: **View Generalization.** We evaluated the view generalization capability of our model in a new environment, which we designed to be as consistent with the training environment as possible. We tested 3 tasks respectively, and the results under 8 viewing angle changes (the two cameras each correspond to 4 changes). For each specified camera view (e.g. Left Camera -30 degrees), we evaluate each model for 2 trials.

4. Experiments

E. Experiments on Generalization

[Background Generalization]

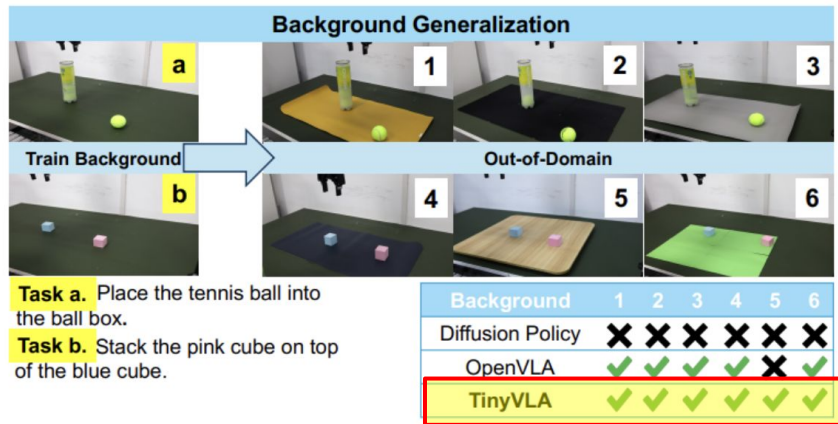


Fig. 6: **Background Generalization.** We utilized six different backgrounds, testing three of them on Task a and the remaining three on Task b. For each background, we evaluate each model for two trials.

[Generalization to different light conditions.]

[Generalization to distractor]

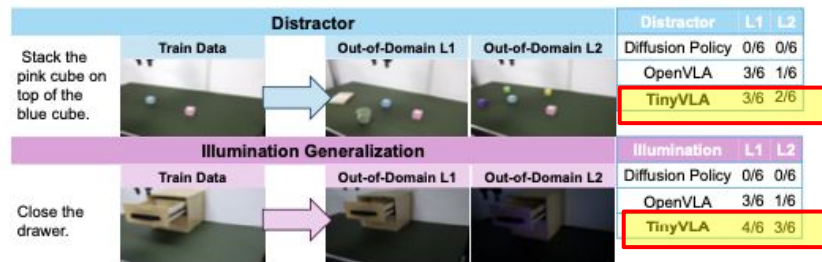


Fig. 7: **Distractor & Illumination Generalization.** For distractor settings, Level L1 involves the addition of objects such as books and cups, which are unrelated to the task. Level L2 involves the inclusion of identical cubes in various colors, adding complexity to the visual environment. For illumination settings, Level L1 represents conditions with reduced lighting, while Level L2 describes scenarios with minimal lighting. For each specified setting (i.e., Distractor L1), we evaluate each model for six trials.

5. Ablation Study

A. Trade-off between size of VLM and TinyVLA’s Performance

Small VLM (0.4B): Often fails due to poor understanding of language instructions

Medium VLM (1.3B): Begins to understand instructions reliably

Large VLM (3B, PaliGemma): Understands instructions well and localizes objects more accurately, leading to significantly fewer failures

B. Choice of Policy Model

Policy Head	PlaceTennis	FlipMug	StackCubes	CloseDrawer	OpenBox
Multi-Layer Perceptron	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0
Action Chunking Transformer	13.3 $\pm$ 0.1	8.3 $\pm$ 0.1	8.3 $\pm$ 0.3	13.3 $\pm$ 0.1	23.3 $\pm$ 0.1
Diffusion Model	90$\pm$0.2	98.3$\pm$0.1	98.3$\pm$0.1	96.7$\pm$0.3	86.7$\pm$0.1

TABLE V: Ablation study on different choices of policy model. We choose TinyVLA-H as our base model and replace the diffusion model with ACT model [46] and vanilla MLP head. We report the success rate on 5 real robot tasks.

C. Which Part of TinyVLA makes it Fast?

Inference Latency on A6000 GPU	
OpenVLA-7B $\rightarrow$ OpenVLA-1B	TinyVLA-1B
292 <i>ms</i> $\rightarrow$ 140 <i>ms</i>	14 <i>ms</i>

TABLE IV: Inference latency is measured quantitatively and reported in milliseconds (ms). The reported values represent the time required for a single action prediction by the OpenVLA-1B/7B and TinyVLA-1B models. Experiments are conducted using a single A6000 GPU.

The improvement does not come solely from using a smaller VLM, but from avoiding autoregressive action-token generation in OpenVLA and instead predicting actions with a diffusion-based approach.

6. Conclusion

- Conventional VLAs are bottlenecked by **autoregressive action tokens** and reliance on **large-scale robot pretraining**.
- **TinyVLA** uses a **lightweight VLM + LoRA ($\approx 5\%$)** to greatly reduce training cost.
- A **diffusion policy head** predicts continuous actions, reducing latency while maintaining performance.
- Results on **simulated and real robots** validate fast, data-efficient VLA.

Thank you